

SIMATS ENGINEERING

ASSESSMENT TOOL 2

Case study

COURSE NAME: Computer Networks

COURSE CODE: CSA0704

COURSE OUTCOME COVERED

CO3: *Analyze the performance of core network protocols such as TCP, UDP, HTTP, and DNS under varying conditions*

Assessment Tool Weightage: 40%

Case study

QUESTIONS:4

Total Marks:40

1.A multinational IT company uses a cloud-based video conferencing platform for daily client meetings. During peak business hours, employees experience frequent voice distortion, frozen video frames, and delayed communication. Network monitoring reports indicate an **8% packet loss, 180 ms jitter**, and fluctuating bandwidth utilization. The IT administrator suspects that the issue is related to the transport protocol or application-level buffering strategy.

Questions

1. Analyze whether the problem is primarily caused by the **transport protocol (TCP/UDP)** or the **application layer**.
2. Justify your answer based on the communication requirements of real-time multimedia applications.
3. Recommend suitable protocol-level or application-level improvements to enhance conferencing quality.

2.A multinational software company hosts its web services across multiple continents. Employees frequently report long delays before the application homepage loads. Network analysis shows that the **average DNS lookup time is approximately 800 ms**, significantly affecting user experience. The organization plans to redesign its DNS infrastructure while ensuring continuous service availability during server failures.

Questions

1. Analyze the reasons behind the high DNS resolution time.
2. Propose a suitable DNS architecture using techniques such as **Anycast DNS, DNS pre-fetching, and TTL optimization** to reduce the lookup time below **50 ms**.
3. Evaluate the advantages and limitations of your proposed architecture in terms of performance, scalability, and availability.

3.A cloud service provider delivers applications to millions of users worldwide. During promotional events, users experience intermittent delays in accessing cloud-hosted services. Investigation reveals that DNS queries are taking nearly **800 ms** because all requests are directed to a single primary DNS server. The company wants a highly available DNS solution capable of handling global traffic efficiently.

Questions

- 4. Examine the impact of centralized DNS architecture on application performance.
- 5. Design a distributed DNS solution that minimizes lookup latency while maintaining high availability.
- 6. Analyze how **TTL values**, **Anycast routing**, and **DNS caching** influence system performance and fault tolerance.

4.A financial institution operates an electronic stock trading platform where thousands of buy and sell orders are submitted every second. During market opening hours, the **average order acknowledgment latency increases from 1 ms to 40 ms**, resulting in delayed trade confirmations and customer dissatisfaction. Network engineers observe increased TCP retransmissions, longer connection establishment times, and excessive buffering.

Questions

- 1. Analyze three possible TCP-related factors responsible for the latency increase, considering **flow control**, **Nagle's algorithm**, and **SYN queue limitations**.
- 2. Explain how each factor affects transaction processing in high-frequency systems
- 3. Recommend appropriate TCP configuration changes to reduce latency and improve transaction reliability.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

1. A multinational IT company uses a cloud-based video conferencing platform for daily client meetings. During peak business hours, employees experience frequent voice distortion, frozen video frames, and delayed communication. Network monitoring reports indicate an **8% packet loss, 180 ms jitter**, and fluctuating bandwidth utilization. The IT administrator suspects that the issue is related to the transport protocol or application-level buffering strategy.

Questions

- Analyze whether the problem is primarily caused by the **transport protocol (TCP/UDP)** or the **application layer**.
- Justify your answer based on the communication requirements of real-time multimedia applications.
- Recommend suitable protocol-level or application-level improvements to enhance conferencing quality.

Answer:

1. Analyze whether the problem is primarily caused by the transport protocol (TCP/UDP) or the application layer

The problem is **primarily associated with transport/network performance, particularly packet loss and high jitter, but the application layer can significantly amplify or mitigate its effects**. It would therefore be inaccurate to blame only TCP/UDP or only the application.

The strongest evidence is the measured **8% packet loss, 180 ms jitter, and fluctuating bandwidth utilization**. These are network-performance conditions that directly affect real-time media delivery. The observed symptoms—**voice distortion, frozen video frames, and delayed communication**—are consistent with packets being lost, arriving irregularly, or arriving too late for playback.

TCP's possible contribution

If the conferencing media were transported using TCP, packet loss could trigger **retransmissions and ordered delivery**. TCP attempts to provide reliable delivery, but retransmitting a packet that is already late may not be useful for a live conversation.

For example:

Media packet sent → packet lost → TCP detects loss → retransmission → packet arrives late → playback is delayed

This can contribute to increased latency and freezing, particularly when packet loss is already high.

However, the case does **not explicitly state whether the conferencing platform is currently using TCP or UDP**. Therefore, it is not technically justified to conclude that TCP is definitely the cause.

UDP's possible contribution

UDP is generally more appropriate for time-sensitive audio/video because it does not impose TCP-style retransmission and ordered-delivery mechanisms. This allows media to continue flowing even when individual packets are lost.

However, UDP does not guarantee delivery. Therefore:

UDP → lower protocol-induced delay, but packet loss must be handled by the application

Consequently, switching to UDP alone would **not eliminate the 8% packet loss or 180 ms jitter**. The underlying network conditions and application behavior must also be addressed.

Application-layer contribution

The application layer is particularly important because the case specifically raises the possibility of an **application-level buffering strategy**.

A conferencing application normally uses buffering to compensate for variation in packet arrival times.

There is an important trade-off:

Small buffer → lower latency but greater risk of interruptions

Large buffer → smoother playback but greater delay

If the buffer is too small relative to the **180 ms jitter**, packets may arrive after their intended playback time, causing interruptions and distortion. If the buffer is excessively large, it can add unnecessary end-to-end delay.

Overall diagnosis

Therefore, the most defensible diagnosis is:

The primary problem is transport/network instability—especially the 8% packet loss and 180 ms jitter—with application-level buffering and media adaptation acting as important secondary factors.

This is a combined problem rather than a single-layer failure.

2. Justify the answer based on the communication requirements of real-time multimedia applications

Real-time multimedia has fundamentally different requirements from applications such as file transfer or email.

For a file transfer:

Correct delivery of every byte is more important than immediate delivery.

For video conferencing:

Timely and continuous delivery is more important than perfect recovery of every packet.

This distinction is critical to the case.

a. Low latency

Video conferencing is interactive. If an audio or video packet arrives significantly late, the conversation may already have progressed.

Therefore, retransmitting an old packet can sometimes be less useful than continuing with

newer media.

The observed **delayed communication** indicates that latency is already affecting the user experience.

b. Low jitter

Jitter is variation in packet arrival time.

The reported **180 ms jitter** means packets are arriving with substantial timing variation. Even if packets are not lost, inconsistent arrival times can cause audio gaps, distortion, or uneven video playback.

A jitter buffer can compensate for this by temporarily holding packets and delivering them to the media decoder at a more regular rate.

However:

More buffering → better jitter absorption → higher latency

Therefore, buffering must be carefully controlled.

c. Tolerance to packet loss

The case reports **8% packet loss**, which is highly problematic for real-time media.

Lost audio packets can cause:

- broken speech,
- missing sounds,
- voice distortion.

Lost video packets can contribute to:

- missing information,
- visual artifacts,
- frozen frames.

Unlike a file-transfer application, a conferencing application cannot always afford to wait indefinitely for retransmission because media has a playback deadline.

d. Adaptive bandwidth usage

The case also reports **fluctuating bandwidth utilization**.

A real-time conferencing application should therefore adapt its media rate to current network conditions.

For example:

High bandwidth → higher video quality

Reduced bandwidth → lower bitrate/resolution/frame rate

This prevents the application from continuously sending more data than the network can effectively deliver.

e. Continuous playback

The objective is not simply to deliver packets successfully; they must be delivered **at an appropriate time for playback**.

This creates the fundamental real-time multimedia principle:

A packet that arrives too late can be almost as harmful as a packet that is lost.

This is why TCP's reliability mechanisms can sometimes conflict with real-time requirements, while UDP requires the application to take greater responsibility for dealing with loss and jitter.

3. Recommend suitable protocol-level or application-level improvements

Because the case involves **8% packet loss, 180 ms jitter, fluctuating bandwidth, voice distortion, frozen video, and delayed communication**, the best solution is a **combined transport/network and application-level strategy**.

A. Use a low-latency transport approach for real-time media

Where the conferencing platform supports it, real-time audio/video should use a **UDP-based media transport approach** rather than relying on TCP retransmission for time-sensitive media.

This reduces delay caused by:

- retransmission,
- strict ordering,
- waiting for lost packets.

However, UDP should be combined with application-level mechanisms for loss and congestion handling.

Benefit: lower latency and better suitability for interactive communication.

B. Implement adaptive jitter buffering

The conferencing application should dynamically adjust its jitter buffer according to current network conditions.

With **180 ms jitter**, a fixed small buffer may cause frequent playback interruptions.

An adaptive approach can respond to changing conditions:

Jitter increases → buffer adjusts upward

Network stabilizes → buffer can reduce

The goal is to absorb timing variation without introducing excessive conversational delay.

C. Implement adaptive bitrate control

The application should continuously monitor bandwidth conditions and reduce its media bitrate when network capacity decreases.

It can dynamically modify:

- video bitrate,
- resolution,
- frame rate,

- encoding quality.

For example:

Congestion detected → reduce bitrate → reduce bandwidth demand → improve continuity

This directly addresses the case's fluctuating bandwidth utilization.

D. Use packet-loss concealment

The application should use **packet-loss concealment/error-resilience techniques** so that isolated missing packets do not immediately result in severe audio or video degradation.

For audio, the application may use information from surrounding audio samples to make short losses less noticeable.

For video, appropriate error-resilience techniques can reduce the visual impact of missing data.

This is particularly valuable because the case reports **8% packet loss**.

E. Prioritize real-time traffic using QoS

At the network level, conferencing traffic should receive appropriate **Quality of Service (QoS)** treatment.

For example:

Voice/video traffic → high priority

Normal web traffic → normal priority

Background downloads → lower priority

This can reduce queueing and help protect real-time media during peak business hours.

F. Investigate and reduce the underlying packet loss

Application techniques cannot fully compensate for sustained **8% packet loss**.

The organization should investigate:

- network congestion,
- overloaded links,
- wireless interference where applicable,
- faulty network devices,
- inadequate bandwidth,
- problematic routing paths.

The objective should be to **reduce packet loss at the network level**, rather than simply hiding its effects through larger buffers.

G. Monitor performance continuously

The organization should monitor:

- packet-loss percentage,
- jitter,
- round-trip latency,
- available bandwidth,
- retransmissions,
- buffer occupancy,
- media bitrate.

This allows the conferencing platform to distinguish between a **network problem** and an **application buffering problem**.

Recommended Solution

Problem	Recommended improvement	Expected effect
8% packet loss	Investigate congestion + packet-loss concealment	Less distortion/freezing
180 ms jitter	Adaptive jitter buffer	Smoother playback
Fluctuating bandwidth	Adaptive bitrate	Prevents excessive bandwidth demand
Real-time delay	UDP-based media transport	Reduces retransmission-related delay
Peak-hour congestion	QoS prioritization	Protects voice/video traffic
Excessive buffering	Dynamically optimize buffer	Balances smoothness and latency
Network instability	Continuous monitoring	Faster identification and correction

Conclusion

The evidence indicates that the conferencing problem is **primarily driven by transport/network conditions—especially the 8% packet loss and 180 ms jitter—while the application-layer buffering strategy determines how strongly users experience those conditions.**

For real-time multimedia, **low latency, low jitter, continuous playback, and adaptive bandwidth usage are more important than guaranteeing retransmission of every packet.** Consequently, a TCP-oriented reliability strategy can be unsuitable for time-sensitive media when retransmissions introduce additional delay.

The recommended solution is therefore a **combined approach**: use a suitable low-latency/UDP-based media transport, reduce the underlying packet loss, implement adaptive jitter buffering and bitrate control, use packet-loss concealment, and prioritize conferencing traffic with QoS.

The key design principle is: deliver media on time rather than merely guaranteeing that every packet eventually arrives. This approach directly addresses the measured network problems and provides a practical way to improve conferencing quality during peak business hours.

2. A multinational software company hosts its web services across multiple continents. Employees frequently report long delays before the application homepage loads. Network analysis shows that the **average DNS lookup time is approximately 800 ms**, significantly affecting user experience. The organization plans to redesign its DNS infrastructure while ensuring continuous service availability during server failures.

Questions

- Analyze the reasons behind the high DNS resolution time.
- Propose a suitable DNS architecture using techniques such as **Anycast DNS**, **DNS pre-fetching**, and **TTL optimization** to reduce the lookup time below **50 ms**.
- Evaluate the advantages and limitations of your proposed architecture in terms of performance, scalability, and availability.

Answer:

The case describes a multinational software company whose web services are hosted across multiple continents. Employees experience long delays before the homepage loads, with an average **DNS lookup time of approximately 800 ms**. The organization therefore needs to redesign DNS while maintaining service availability during server failures. The assessment specifically asks for **Anycast DNS**, **DNS pre-fetching**, and **TTL optimization**, with a target of reducing lookup latency to **below 50 ms**.

1. Analyze the reasons behind the high DNS resolution time

The reported **800 ms DNS lookup time** indicates that DNS resolution has become a significant component of the application's initial access delay. Since the company operates across multiple continents, the problem is likely related to a combination of **DNS server location**, **network round-trip time**, **repeated DNS resolution**, **cache effectiveness**, and **lack of geographically distributed DNS service**.

a. Geographic distance

If users in different continents must query DNS infrastructure located far away, DNS requests and responses travel over longer network paths.

For example:

User → Distant DNS server → DNS response → User

The longer physical and network distance increases round-trip time. Consequently, a DNS architecture that does not bring DNS service closer to users can create substantial latency.

b. Lack of geographically distributed DNS service

A multinational organization needs DNS infrastructure capable of serving users from different geographical regions efficiently.

If DNS queries are concentrated on a small number of locations, users may experience:

- longer network paths,
- higher latency,
- increased server workload,
- greater dependency on particular DNS locations.

This is particularly important because the case identifies an **800 ms average lookup time** and requires the redesigned architecture to support continuous availability.

c. DNS cache misses

DNS caching normally allows a resolver to answer a request using a previously stored DNS record instead of performing another complete resolution.

When a cache entry is unavailable or has expired:

Cache miss → Recursive resolution → Additional DNS communication → Higher latency

Frequent cache misses can therefore increase average DNS lookup time.

d. Poor TTL configuration

The **Time to Live (TTL)** value determines how long DNS information can remain cached.

If TTL values are too short, DNS records expire frequently:

Short TTL → Frequent expiration → More DNS queries → More resolution overhead

This can increase DNS traffic and lookup latency.

However, an excessively long TTL also has disadvantages because changes to DNS records take longer to propagate. Therefore, TTL must balance **performance and flexibility**.

e. Lack of DNS pre-fetching

If frequently accessed DNS records are not refreshed before expiration, the first user request after cache expiration may experience a cache miss.

DNS pre-fetching can reduce this problem by refreshing frequently used records before they become unavailable from cache.

Thus:

Without pre-fetching:

Cache expires → User request → DNS resolution → Delay

With pre-fetching:

Record refreshed in advance → User request → Cached response → Lower delay

f. Multiple DNS resolution steps

When a resolver does not have the required record cached, it may need to perform multiple resolution steps before obtaining the authoritative answer.

Each additional network interaction introduces potential latency.

Therefore, the 800 ms figure may represent not only DNS-server processing but also the accumulated effects of **network distance, cache misses, and recursive resolution**.

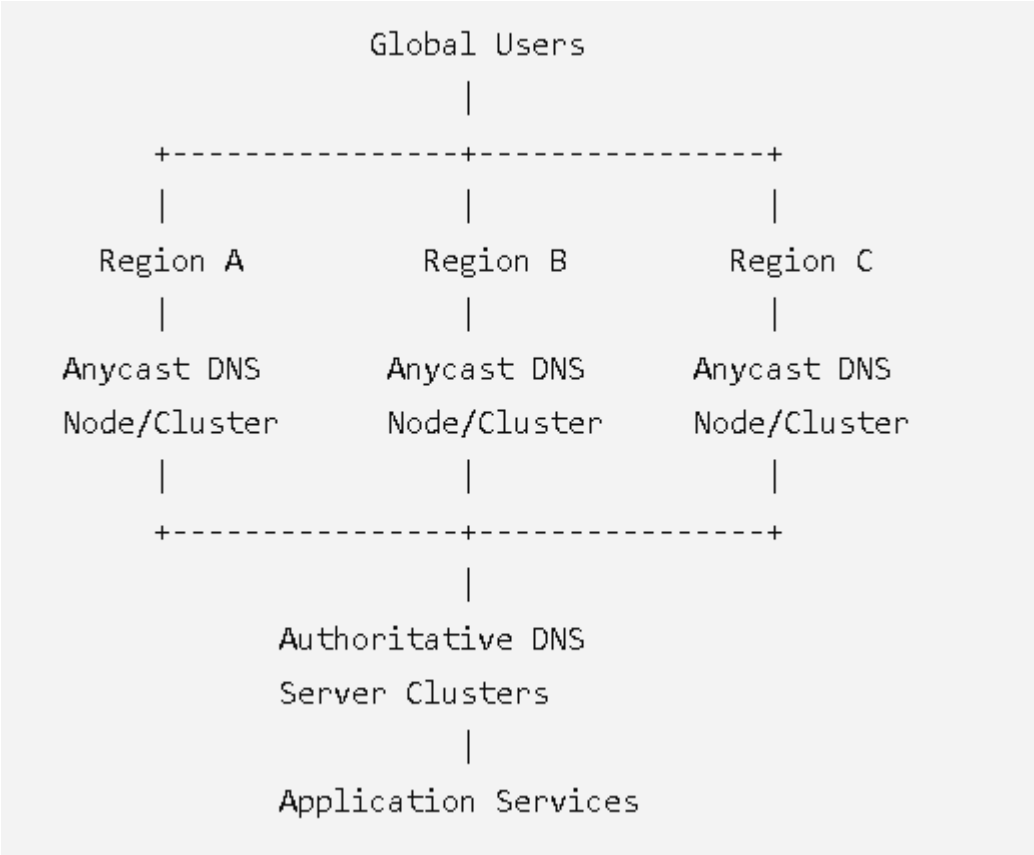
Overall diagnosis

The high DNS latency is therefore best viewed as a **distributed DNS-performance problem rather than simply a slow DNS server**.

2. Propose a suitable DNS architecture using Anycast DNS, DNS pre-fetching, and TTL optimization to reduce lookup time below 50 ms

The recommended architecture should distribute DNS service across multiple geographical locations and ensure that users are directed toward an appropriate DNS service point.

Proposed architecture



The architecture should combine **Anycast routing, caching, DNS pre-fetching, and optimized TTL values.**

A. Anycast DNS

With Anycast DNS, multiple DNS servers in different geographical locations advertise the **same IP address/prefix.**

The network routing system directs a user's DNS query toward an appropriate available DNS location.

Conceptually:

User in Asia → nearest/appropriate Anycast DNS node

User in Europe → nearest/appropriate Anycast DNS node

User in North America → nearest/appropriate Anycast DNS node

This reduces the distance between users and DNS service and therefore reduces DNS round-trip latency.

Performance benefit

Instead of:

User → distant DNS server → response

the system aims for:

User → nearby Anycast DNS node → response

This is the primary architectural mechanism for moving the average lookup time toward the

below-50-ms target.

Availability benefit

Anycast also provides redundancy.

If one DNS location fails:

Failed DNS node → routes withdrawn/traffic redirected → another Anycast node handles requests

Therefore, a single DNS server or site failure does not necessarily make the DNS service unavailable.

B. DNS caching

Caching should be used at recursive resolvers and other appropriate DNS layers to avoid repeatedly resolving the same records.

For a cache hit:

Client → Resolver → Cached answer

This is significantly more efficient than performing a complete DNS resolution every time.

Caching reduces:

- DNS query traffic,
- recursive resolution workload,
- network round trips,
- response latency.

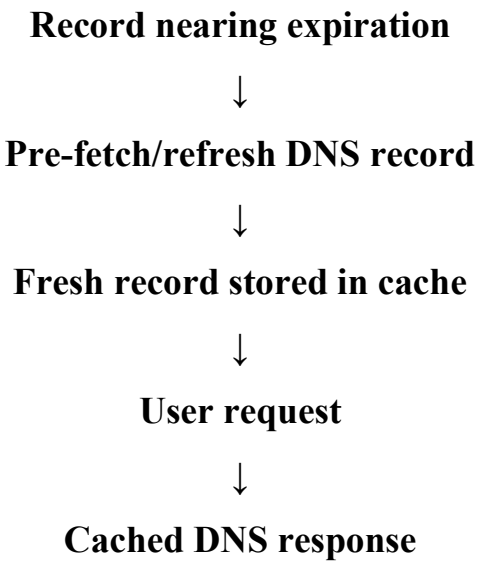
Therefore, effective caching is essential for reaching the **sub-50-ms target**.

C. DNS pre-fetching

The organization should use DNS pre-fetching for frequently accessed records.

When a cached record is approaching expiration, the resolver can refresh it before users encounter a cache miss.

The process becomes:



This minimizes the likelihood that users will experience a delay caused by a cache miss.

D. TTL optimization

TTL values should be selected according to the stability and update frequency of each DNS record.

A balanced strategy is preferable:

Very short TTL → frequent queries → higher DNS load

Very long TTL → fewer queries → but slower DNS change propagation

Therefore, the company should use **appropriate TTL values rather than simply choosing the longest possible TTL.**

For relatively stable records, a longer TTL can improve cache efficiency.

For records that change frequently, a shorter TTL provides greater flexibility.

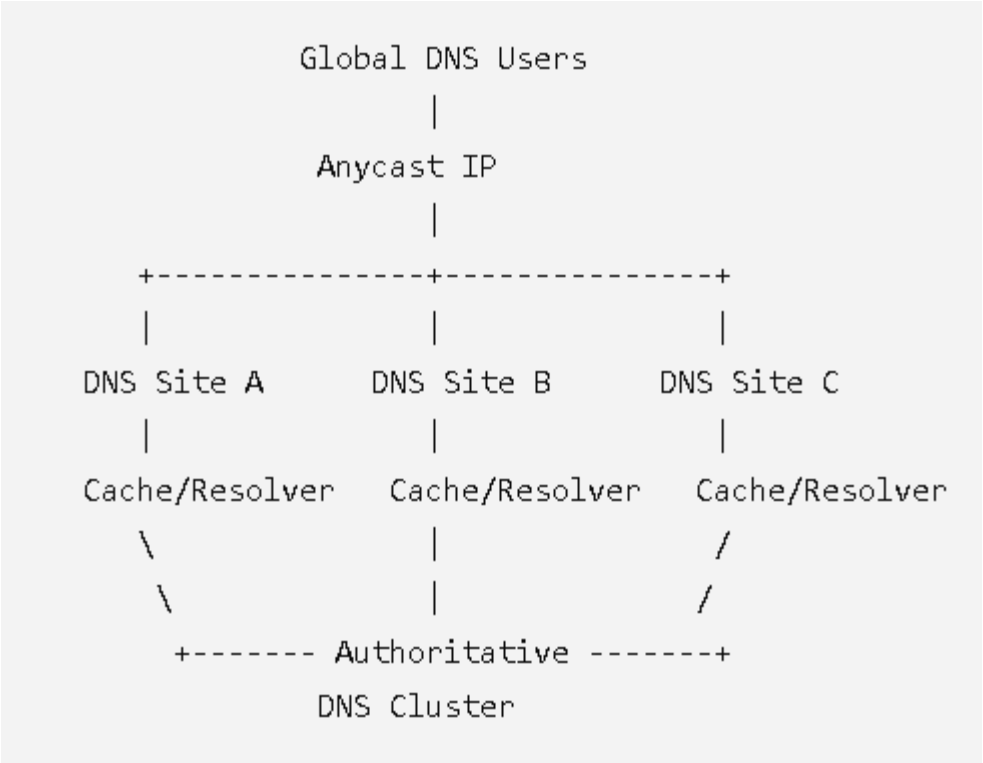
This creates a balance between:

Performance + cache efficiency + change responsiveness

E. DNS redundancy

The architecture should use multiple DNS nodes rather than depending on one server.

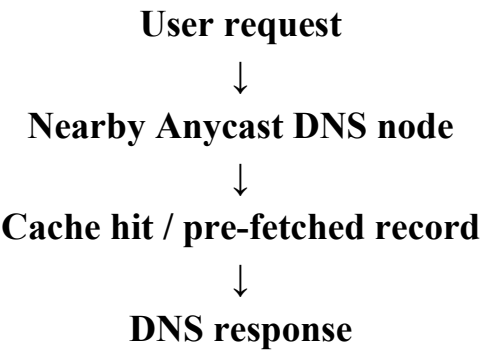
For example:



The authoritative DNS infrastructure should itself be redundant so that failure of one server or site does not interrupt DNS service.

Targeted performance path

The redesigned system should aim for:



This minimizes both **geographic network latency** and **recursive DNS processing**.

The organization should monitor actual latency after deployment and adjust Anycast locations, caching, and TTL policies accordingly rather than assuming that the architecture automatically guarantees a sub-50-ms result.

3. Evaluate the advantages and limitations in terms of performance, scalability, and availability

Performance

Advantages

1. **Anycast reduces geographic distance** between users and DNS service.
2. **Caching eliminates unnecessary recursive lookups.**
3. **DNS pre-fetching reduces cache-miss delays.**
4. **Appropriate TTL values improve cache effectiveness.**
5. Multiple DNS locations reduce the possibility of one overloaded location becoming the only service point.

Together, these mechanisms can substantially reduce DNS lookup latency compared with the reported **800 ms** baseline.

Limitation:

Achieving **below 50 ms** cannot be guaranteed solely by DNS configuration. Actual performance also depends on user location, network conditions, resolver behavior, cache state, and routing.

Scalability

Advantages

Anycast provides a distributed architecture in which DNS traffic can be handled across multiple locations.

Instead of:

Millions of users → One DNS location

the system becomes:

Millions of users → Multiple distributed DNS locations

This allows capacity to be added geographically and reduces dependence on a single infrastructure point.

Caching also reduces the number of queries that need to reach authoritative infrastructure.

Limitation:

Operating many DNS locations increases infrastructure and management complexity. The organization must maintain configuration consistency, monitor each location, and ensure that DNS records are correctly synchronized.

Availability

Advantages

The distributed design provides strong fault tolerance.

If one Anycast DNS location fails:

Failure → Route withdrawal/failover → Users redirected to another DNS location

Similarly, multiple authoritative DNS servers prevent a single server failure from taking down the complete DNS service.

This directly supports the organization's requirement for **continuous service availability during server failures**.

Limitation:

Anycast does not eliminate every failure scenario. Incorrect routing, configuration errors, widespread network failures, or problems affecting multiple DNS locations can still affect availability.

TTL trade-off

TTL configuration illustrates an important design trade-off:

TTL strategy	Performance	Availability/Flexibility
Very short TTL	More DNS queries and potentially higher load	Fast propagation of changes
Very long TTL	Excellent caching and lower query load	Changes/failures may take longer to propagate
Optimized TTL	Balanced performance	Balanced responsiveness

Therefore, TTL values should be selected according to the characteristics of individual DNS records.

Overall Evaluation

Requirement	Proposed mechanism	Main benefit	Main limitation
Reduce lookup latency	Anycast DNS	Brings DNS service closer to users	More infrastructure complexity
Reduce repeated resolution	DNS caching	Faster cache-hit responses	Cache may become stale
Avoid cache-miss delay	DNS pre-fetching	Refreshes records proactively	Additional DNS traffic/processing
Balance cache and updates	TTL optimization	Controls cache duration	Requires careful tuning
Maintain service during failure	Multiple Anycast/authoritative nodes	Removes single-server dependency	More operational complexity

Conclusion

The **800 ms DNS lookup time** is a significant contributor to the application's poor user experience. The multinational nature of the organization makes a geographically distributed DNS architecture particularly important.

The recommended solution is a **distributed DNS architecture based on Anycast DNS, effective DNS caching, DNS pre-fetching, optimized TTL values, and redundant authoritative DNS servers**. Anycast reduces the network distance to DNS service, caching minimizes repeated resolution, pre-fetching reduces cache-miss delays, and TTL optimization balances performance against the need for DNS changes to propagate.

The architecture also improves scalability because DNS requests can be distributed across multiple locations, while redundancy improves availability when an individual DNS server or site fails.

The target of reducing DNS lookup time from approximately 800 ms to below 50 ms should be treated as an engineering objective that must be validated through monitoring and measurement. The proposed architecture provides the mechanisms needed to achieve that improvement while simultaneously strengthening scalability and fault tolerance.

This approach addresses all three Level-4 rubric dimensions relevant to the case: **deep application of DNS concepts, strong analysis of performance/scalability/availability trade-offs, and a feasible architecture with clear justification**. The assessment rubric specifically rewards thorough case understanding, effective concept application, in-depth reasoning, and feasible solutions

3.A cloud service provider delivers applications to millions of users worldwide. During promotional events, users experience intermittent delays in accessing cloud-hosted services. Investigation reveals that DNS queries are taking nearly **800 ms** because all requests are directed to a single primary DNS server. The company wants a highly available DNS solution capable of handling global traffic efficiently.

Questions

- Examine the impact of centralized DNS architecture on application performance.
- Design a distributed DNS solution that minimizes lookup latency while maintaining high availability.
- Analyze how **TTL values**, **Anycast routing**, and **DNS caching** influence system performance and fault tolerance.

Answer:

The cloud service provider's centralized DNS architecture is a **major performance bottleneck** because millions of users worldwide are sending DNS requests to a **single primary DNS server**. During promotional events, DNS queries take nearly **800 ms**, resulting in intermittent delays when users access cloud-hosted services. The case therefore demonstrates that the DNS infrastructure itself can become a limiting factor even when the underlying cloud applications are capable of handling large numbers of users.

1. DNS becomes a single point of failure

The most serious weakness of the centralized architecture is that the primary DNS server represents a **single point of failure**.

If this server experiences:

- hardware failure,
- software failure,
- network failure,
- overload, or
- maintenance,

users may be unable to resolve the cloud service's domain name.

Therefore, even if the actual application servers remain operational, users may effectively be unable to access them because DNS resolution is unavailable.

DNS failure → Domain cannot be resolved → Application becomes inaccessible

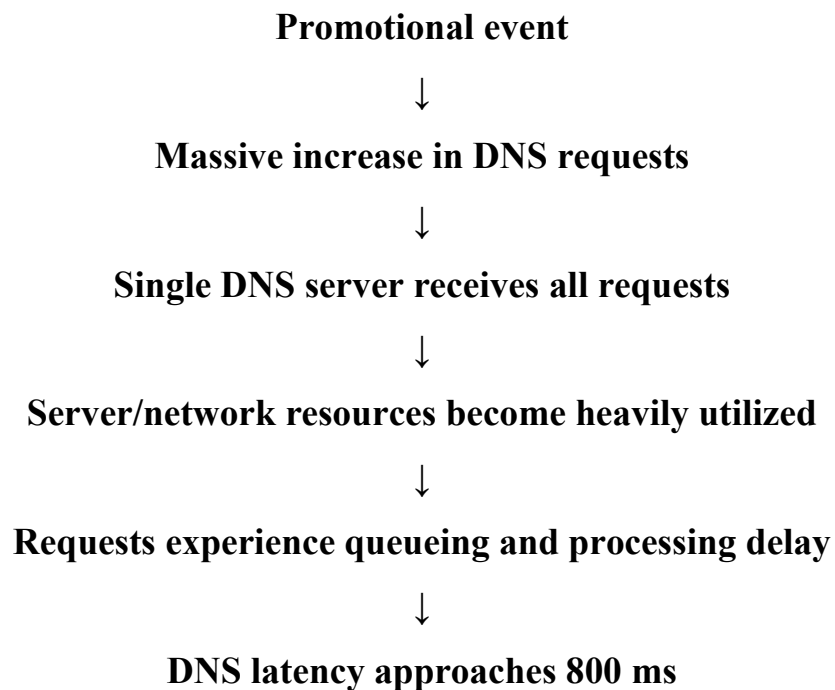
This directly conflicts with the company's requirement for a **highly available DNS solution**.

2. Centralization creates a DNS processing bottleneck

The case states that **all DNS requests are directed to one primary DNS server**.

During promotional events, the number of users can increase dramatically. Consequently, a single server must process a very large number of queries simultaneously.

The resulting process can be represented as:



This explains why users experience intermittent delays specifically during high-demand periods.

3. 800 ms DNS latency directly increases application access time

DNS resolution normally occurs before the client can establish communication with the requested cloud service.

A simplified access sequence is:

User requests application

→ **DNS resolution**

→ **Connection to cloud service**

→ **Application request**

→ **Application response**

If DNS resolution alone takes approximately **800 ms**, the user has already experienced a substantial delay before the application itself begins responding.

Therefore, the perceived application latency is not necessarily caused by slow application processing. **DNS latency becomes part of the user's total application access time.**

This is particularly important for cloud services where users expect rapid access during high-traffic events.

4. Global users increase the effect of centralization

The service is used by **millions of users worldwide**.

If a single DNS server is responsible for global requests, users located far from that server may experience additional network round-trip latency.

For example:

User in Asia → Central DNS server → DNS response

or

User in Europe → Central DNS server → DNS response

Instead of obtaining DNS service from a nearby location, every user depends on the same centralized infrastructure.

This makes geographical distance and network conditions part of the DNS performance problem.

5. Poor scalability during promotional events

A centralized architecture has limited ability to scale horizontally.

With increasing demand:

More users → More DNS queries → More workload on the same server

The provider cannot efficiently distribute the DNS workload across independent geographical locations if all requests continue to target one primary server.

This creates a mismatch between the scalability of the cloud application and the scalability of its DNS infrastructure.

The application may have thousands of servers available globally, but if users must first communicate with one overloaded DNS server, DNS can become the **first bottleneck in the access path**.

6. Increased risk of timeout and repeated queries

When a DNS server becomes overloaded, responses may take longer to return. If a client or resolver does not receive a response within an expected period, it may retry the query.

This can create additional traffic:

High load → Delayed response → Retry → More DNS traffic → Greater load

This feedback effect can make an already overloaded DNS infrastructure perform even worse.

Therefore, centralization can create both a **capacity problem** and a **reliability problem**.

7. Impact on user experience

The technical DNS problem ultimately becomes a business problem.

The reported **800 ms DNS delay** can lead to:

- slower initial application access,
- intermittent delays,
- poor responsiveness during promotional events,
- customer dissatisfaction,
- increased risk of service unavailability.

Since the company provides services to millions of users, even a relatively small delay can affect a very large number of requests simultaneously.

Centralized DNS vs Distributed DNS

Factor	Centralized DNS	Distributed DNS
DNS processing	Concentrated at one primary server	Distributed across multiple locations
Geographic latency	Potentially high	Can be reduced using geographically closer nodes
Scalability	Limited during traffic spikes	Better distribution of global traffic
Fault tolerance	Low; primary server is a critical failure point	High; multiple DNS nodes provide redundancy
Promotional traffic	Can overload one server	Load can be distributed
Availability	Vulnerable to server failure	Can continue through alternate DNS locations
User experience	Can produce high lookup latency	Lower and more consistent lookup latency

Overall Impact on Application Performance

The centralized DNS architecture affects the cloud service at **three major levels**:

Performance

The **800 ms DNS lookup time** adds significant delay before users can access the application.

Scalability

Millions of global users and promotional traffic spikes concentrate demand on one DNS server, creating a bottleneck.

Availability

The primary DNS server is a **single point of failure**, meaning its failure can prevent users from resolving the cloud service even when the application infrastructure itself is still functioning.

Conclusion

The centralized DNS architecture is a **critical bottleneck for the cloud provider's application performance**. The case explicitly identifies that all DNS requests are directed to a single primary server and that DNS queries are taking nearly **800 ms** during promotional events.

The architecture therefore suffers from **centralized processing, poor scalability during traffic spikes, increased geographic latency, and a single point of failure**. These problems directly increase application access time and can reduce service availability.

The appropriate solution is to replace the centralized design with a **distributed, geographically redundant DNS architecture** capable of handling global queries across multiple DNS locations. This will be developed in **Question 2**, where Anycast and distributed DNS nodes can be used to reduce lookup latency while maintaining high availability.

Rubric focus: This answer identifies the key case issues, applies DNS concepts directly to the observed 800 ms delay, explains cause-and-effect relationships, and connects performance, scalability, and availability—matching the Level 4 emphasis on thorough case understanding and in-depth analysis.

4.A financial institution operates an electronic stock trading platform where thousands of buy and sell orders are submitted every second. During market opening hours, the **average order acknowledgment latency increases from 1 ms to 40 ms**, resulting in delayed trade confirmations and customer dissatisfaction. Network engineers observe increased TCP retransmissions, longer connection establishment times, and excessive buffering.

Questions

- Analyze three possible TCP-related factors responsible for the latency increase, considering **flow control, Nagle's algorithm, and SYN queue limitations**.
- Explain how each factor affects transaction processing in high-frequency systems
- Recommend appropriate TCP configuration changes to reduce latency and improve transaction reliability.

Answer:

In an electronic stock trading platform, thousands of buy and sell orders may arrive every second, especially during market opening hours. The increase in order acknowledgment latency from **1 ms to 40 ms**, together with TCP retransmissions, longer connection establishment times, and excessive buffering, indicates TCP-level performance problems. In a high-frequency trading environment, even a few milliseconds of additional delay can affect order execution, trade confirmation, and customer satisfaction.

Three important TCP-related factors that can contribute to this latency increase are **TCP flow control and receive-window limitations, Nagle's algorithm, and SYN queue limitations**.

1. Three Possible TCP-Related Factors Responsible for the Latency Increase

A. TCP Flow Control and Receive-Window Limitations

TCP uses a **sliding-window mechanism** to control how much data a sender can transmit before receiving acknowledgments. The receiver advertises a **receive window (rwnd)** indicating the amount of buffer space available.

If the receiving application processes messages slowly or the TCP receive buffer becomes full, the advertised window becomes smaller. This can force the sender to reduce its transmission rate or temporarily stop sending.

In the stock trading system, thousands of order messages and acknowledgments can arrive simultaneously. If the TCP receive buffers are too small, excessive buffering and window reduction can occur.

Impact:

- Sender may be forced to wait for acknowledgments.
- Throughput decreases during traffic bursts.
- Queuing delay increases.
- TCP may experience zero-window or near-zero-window conditions.
- Order acknowledgments take longer to reach applications.

Thus, inadequate TCP buffer/window configuration can contribute significantly to the increase from **1 ms to 40 ms** latency.

B. Nagle's Algorithm

Nagle's algorithm is designed to reduce network overhead by combining small TCP segments before transmission. It generally works by delaying small packets until either enough data is accumulated or an acknowledgment is received.

Although this improves network efficiency, it can introduce additional delay for applications that frequently send **small, latency-sensitive messages**.

Stock trading systems typically exchange small messages such as:

- Buy orders
- Sell orders
- Order acknowledgments
- Execution confirmations
- Status updates

If Nagle's algorithm is enabled, a small order or acknowledgment may be temporarily buffered instead of being transmitted immediately.

This buffering can be especially harmful in high-frequency trading because the system prioritizes **minimum latency rather than maximum packet efficiency**.

C. SYN Queue Limitations

Before a TCP connection can transfer application data, the TCP three-way handshake occurs:

Client → SYN → Server

Server → SYN-ACK → Client

Client → ACK → Server

The server maintains a **SYN queue** for connections that are in the process of being established. During market opening, a sudden burst of trading clients or connections can generate a large number of SYN requests. If the SYN backlog/queue is too small, the queue can become full.

This may result in:

- Dropped SYN requests
- Retransmitted SYN packets
- Longer connection establishment times
- Connection delays
- Increased CPU/network overhead

The observed **longer connection establishment time** is therefore a strong indication that SYN backlog limitations may be contributing to the problem.

2. Effect of Each Factor on Transaction Processing in High-Frequency Systems

TCP Factor	Effect on Transaction Processing	Impact on HFT
Flow control / small receive window	Limits amount of outstanding data and causes sender to wait	Increases queuing and acknowledgment delay
Nagle's algorithm	Buffers small TCP segments before transmission	Adds unnecessary milliseconds to order/confirmation messages
SYN queue limitation	Delays or drops connection requests during bursts	Increases connection setup time and may cause retransmissions

Flow Control

In high-frequency trading, orders are generated in very short intervals. A small receive window can prevent the sender from continuously transmitting order-related data. Once the receive buffer fills, TCP applies backpressure.

For example, if hundreds of order messages arrive simultaneously but the receiver cannot process them quickly enough, packets can remain queued. This increases **end-to-end latency** and can cause delayed trade confirmations.

Nagle's Algorithm

High-frequency applications often send many small messages. Nagle's algorithm may combine these messages to reduce the number of TCP segments.

However, the resulting waiting period is undesirable in financial transactions. A buy order that is delayed by buffering may reach the trading engine later than competing orders.

Therefore, in latency-sensitive systems, **immediate transmission of small messages is generally more important than reducing packet count.**

SYN Queue Limitations

If a new TCP connection is required for trading requests and the SYN queue becomes saturated, clients may experience repeated connection attempts.

For example:

Connection burst → SYN queue fills → SYN dropped → client retransmits SYN → connection establishment delayed

This increases transaction setup time and can also increase network traffic due to retransmissions.

In a high-frequency environment, repeated connection establishment should generally be minimized because maintaining persistent connections is more efficient.

3. Recommended TCP Configuration Changes

1. Increase TCP Receive and Send Buffers

The operating system should be configured with appropriately sized TCP receive and send buffers.

Increasing these values allows the system to handle temporary bursts of order traffic without immediately exhausting the TCP window.

Recommendation:

- Increase TCP receive buffer limits.
- Increase TCP send buffer limits where required.
- Monitor rwnd and application receive rates.
- Use TCP window scaling for high-throughput connections.
- Tune buffers based on actual traffic rather than setting unnecessarily large values.

This reduces flow-control-induced waiting and improves throughput during market-opening bursts.

2. Disable Nagle's Algorithm for Latency-Sensitive Connections

For trading connections that exchange small, time-sensitive messages, disable Nagle's algorithm using the **TCP_NODELAY** socket option.

Conceptually:

```
setsockopt(socket, IPPROTO_TCP, TCP_NODELAY, 1)
```

This allows small TCP messages to be transmitted immediately instead of waiting for additional data.

Benefit:

Without TCP_NODELAY:

Small order → buffering → transmission → acknowledgment

With TCP_NODELAY:

Small order → immediate transmission → faster acknowledgment

This can significantly reduce application-level latency for small transaction messages.

However, TCP_NODELAY should be applied selectively because disabling Nagle's algorithm can increase the number of packets and network overhead.

3. Increase SYN Backlog / SYN Queue Capacity

The server should be configured with a sufficiently large SYN backlog to handle sudden connection bursts.

Appropriate operating-system TCP parameters should be tuned so that the server can maintain more pending connection requests.

Additionally, **SYN cookies** can be enabled as a protection mechanism when the system

experiences SYN-flood-like conditions.

This helps prevent legitimate connections from being rejected when the backlog becomes temporarily saturated.

4. Prefer Persistent TCP Connections

For a high-frequency trading platform, clients should preferably maintain **long-lived TCP connections** instead of establishing a new connection for every order.

This avoids repeated:

SYN → SYN-ACK → ACK

handshakes and therefore reduces connection establishment latency.

Persistent connections are particularly useful during market opening, when thousands of transactions can be generated within a very short period.

5. Reduce Unnecessary TCP Retransmissions

Since the network engineers have already observed increased TCP retransmissions, the system should also investigate:

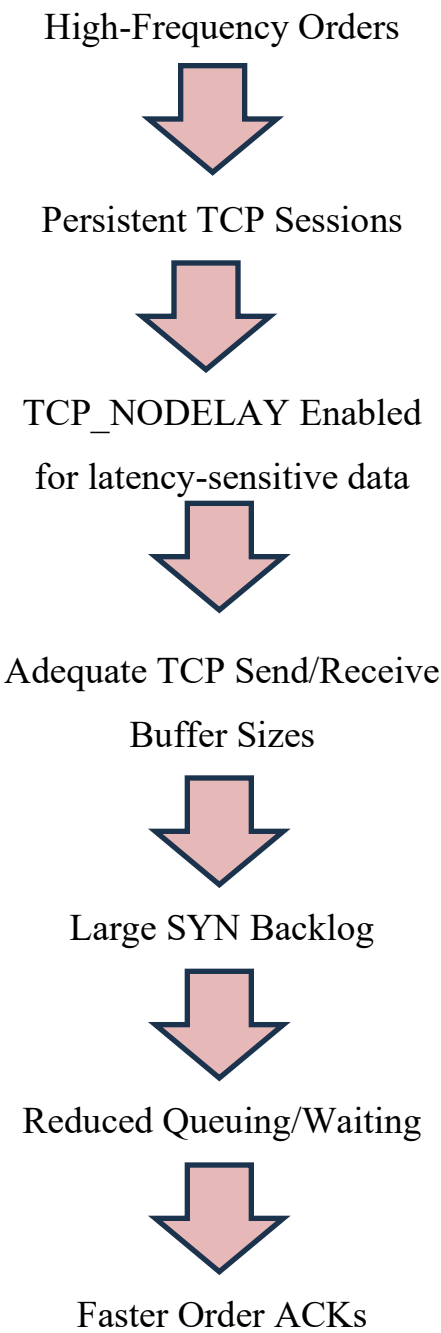
- Packet loss
- Network congestion
- NIC performance
- Buffer overflow
- Network interface errors
- Server CPU saturation
- Queue drops

TCP configuration alone cannot eliminate retransmissions caused by an underlying packet-loss problem.

Therefore, network monitoring should be combined with TCP tuning.

Recommended Configuration Strategy

A practical approach for the trading platform is:



Conclusion

The increase in acknowledgment latency from **1 ms to 40 ms** can be associated with multiple TCP-level bottlenecks. **Flow-control limitations** can cause sender blocking and excessive buffering, **Nagle's algorithm** can delay small latency-sensitive trading messages, and **SYN queue limitations** can increase connection establishment time during sudden traffic bursts.

For a high-frequency trading platform, the recommended approach is to **appropriately increase TCP buffer/window limits, disable Nagle's algorithm using TCP_NODELAY for latency-sensitive connections, increase the SYN backlog, use persistent TCP connections, and investigate the underlying causes of retransmissions.**

These changes can reduce unnecessary TCP waiting, improve order acknowledgment speed, handle market-opening bursts more effectively, and increase the reliability of transaction processing. Most importantly, TCP tuning should be combined with **application-level optimization and network monitoring**, because achieving consistently low latency requires the entire communication path to be optimized.